

Laporan Baseline Tahap 0

Dokumen ini adalah artefak awal untuk Tahap 0 - Audit Baseline dan Reproducibility.

Tujuan

Tahap 0 bertujuan memastikan baseline CDE-Mapper dapat dijalankan ulang secara konsisten sebelum adaptasi Bahasa Indonesia dilakukan. Fokus tahap ini adalah audit struktur, konfigurasi, dependency, input smoke test, dan kesiapan eksekusi.

Artefak yang Dibuat

- `configs/baseline.yaml`: konfigurasi deklaratif baseline dan command inference.
- `data/input/baseline_smoke.csv`: input kecil untuk smoke test loader custom.
- `scripts/baseline_smoke.py`: audit lokal tanpa network/LLM untuk import modul, loader, JSON template, dan reservoir SQLite sementara.
- `scripts/audit_baseline_wsl.sh`: wrapper WSL yang menjalankan audit dengan conda env `cde-mapper`.
- `Riset/LAPORAN_BASELINE.md`: laporan audit tahap 0.

Environment

Sesuai permintaan, audit ditargetkan berjalan di WSL dengan conda environment:

```
conda activate cde-mapper
```

File `environment.yml` sengaja tidak digunakan.

Command wrapper:

```
wsl -e bash -lc "cd /mnt/d/Program/cde_mapper && bash scripts/audit_baseline_wsl.sh"
```

Jika conda tidak otomatis tersedia pada shell WSL non-interaktif, jalankan dari terminal WSL:

```
cd /mnt/d/Program/cde_mapper
source ~/miniconda3/etc/profile.d/conda.sh
conda activate cde-mapper
python scripts/baseline_smoke.py
```

Baseline Runner Saat Ini

Entry point utama baseline:

```
python run.py
```

Command inference minimal untuk data custom:

```
python run.py \
  --flag inference \
  --input_file data/input/baseline_smoke.csv \
  --custom_data \
  --is_omop_data \
  --collection_name concept_mapping_1 \
  --llm_id google/gemma-3n-E4B-it \
```

```
--topk 5 \  
--output_file data/output/baseline_smoke_mapped.csv
```

Catatan: command inference penuh membutuhkan koneksi ke Qdrant/Athena dan runtime/kredensial LLM yang sesuai. Smoke test Tahap 0 tidak memanggil network atau LLM.

Dependency Aktual

Sumber dependency yang ditemukan:

- `pyproject.toml`: dependency proyek Poetry, termasuk `langchain`, `qdrant-client`, `fastembed`, `torch`, `transformers`, `pandas`, `rapidfuzz`, dan provider LLM.
- `requirements.txt`: lock/hasil pip-compile yang lebih panjang dan tampaknya berasal dari baseline upstream.

Catatan audit:

- `README.md` masih menyebut `requirements.in`, tetapi file yang tersedia di repo saat ini adalah `requirements.txt` dan `pyproject.toml`.
- Untuk reproducibility penelitian, dependency sebaiknya distandarkan ke satu sumber utama. Karena user meminta memakai conda env `cde-mapper` di WSL, status paket aktual perlu diambil dari environment tersebut saat audit dijalankan.

Smoke Test

Smoke test lokal memeriksa:

- import modul baseline inti;
- loader custom CSV terhadap `data/input/baseline_smoke.csv`;
- validitas JSON `data/input/mapping_templates.json`;
- inisialisasi reservoir SQLite sementara dari `database_data`;
- import stack retrieval tanpa memanggil Qdrant/Athena/LLM.

Hasil audit otomatis akan ditulis ke:

`Riset/baseline_audit_result.json`

Hasil eksekusi pada WSL conda env `cde-mapper`:

- status: berhasil;
- Python: 3.10.19 dari conda-forge;
- platform: WSL2 Linux;
- durasi audit: 3.45 detik;
- jumlah query smoke yang berhasil dimuat: 3;
- jumlah row reservoir sementara: 10;
- `data/input/mapping_templates.json` valid dan memiliki `database_data=2063`;
- retrieval stack penuh tidak dipanggil pada smoke test untuk menghindari kebutuhan Qdrant, Athena, GPU, dan LLM.

Temuan Awal

- Baseline sudah memiliki entry point `run.py`, tetapi command di `README` lama masih mengarah ke path `mapping_tool/rag/vector_index.py`; command Tahap 0 mengarahkan kem-

bali ke `run.py`.

- `rag/data_loader.py` memiliki loader custom CSV yang cocok untuk data dictionary lokal dengan kolom `variablename`, `variablelabel`, `categorical`, `units`, `formula`, dan `visits`.
- Reservoir lokal baseline dikelola dengan SQLite melalui `rag/sql.py` dan konfigurasi `DB_FILE = 'variables.db'`.
- File `variables.db` tidak perlu ada sebelum run; database dibuat otomatis saat `DataManager` dijalankan.
- Smoke test sengaja memakai temporary SQLite database agar tidak mengubah reservoir kerja.
- `rag/__init__.py` sebelumnya melakukan wildcard import ke hampir seluruh stack retrieval/LLM. Ini membuat import ringan gagal ketika CUDA device atau API key LLM tidak tersedia. File tersebut sudah dibuat ringan sehingga modul spesifik dapat diimpor secara reproducible.
- Audit dipaksa CPU-only dengan `CUDA_VISIBLE_DEVICES=""`, karena Tahap 0 tidak membutuhkan GPU.

Gap Teknis untuk Tahap Berikutnya

- Standardisasi dependency perlu diputuskan: `pyproject.toml`, `requirements.txt`, atau export dari `conda env cde-mapper`.
- Command inference penuh masih tergantung koneksi Qdrant/Athena dan LLM.
- Belum ada dataset gold Bahasa Indonesia untuk evaluasi mapping final.
- README perlu dipertahankan sinkron dengan entry point aktual jika pipeline berubah.

Implementasi Full Inference dan Evaluasi

Tahap 0 dilengkapi dengan artefak berikut:

- `scripts/audit_baseline_integration.py`: validasi online Qdrant, Athena, dan Together;
- `scripts/baseline_experiment.py`: retrieval dense dan sparse secara berurutan, merger Top-10, latency per sumber, error per query, serta manifest run;
- `scripts/baseline_llm_rerank.py`: reranking kandidat tersimpan menggunakan `google/gemma-3n-E4B-it`;
- `evaluation/baseline_retrieval_eval.py`: accuracy, precision, recall, MRR, NDCG, coverage, error rate, serta distribusi latency;
- `scripts/capture_baseline_environment.py`: snapshot Python, platform, commit, conda package, dan pip package tanpa secret;
- `data/gold/baseline_retrieval_gold.jsonl`: gold subset 10 query dengan OMOP ID dan vocabulary code ekuivalen.

Command reproducible:

```
conda run --no-capture-output -n cde-mapper \
  python scripts/baseline_experiment.py --run-id baseline-gemma3n-stage0 --skip-llm-check

conda run --no-capture-output -n cde-mapper \
  python scripts/baseline_llm_rerank.py \
  --input data/output/baseline/baseline-gemma3n-stage0.jsonl \
  --output data/output/baseline/baseline-gemma3n-stage0-reranked.jsonl

conda run --no-capture-output -n cde-mapper \
```

```
python evaluation/baseline_retrieval_eval.py \
--input data/output/baseline/baseline-gemma3n-stage0-reranked.jsonl \
--output Riset/baseline_reranked_metrics.json
```

--skip-llm-check pada retrieval run hanya mencegah model LLM hidup bersamaan dengan model embedding. Validasi LLM online dan reranking aktual tetap dijalankan sebagai proses terpisah.

Hasil Baseline 10 Juli 2026

Ground Truth dan Objek Perbandingan

Evaluasi menggunakan `data/gold/baseline_retrieval_gold.jsonl` yang berisi 10 query. Setiap query memiliki satu konsep target yang direpresentasikan dalam dua identifier ekuivalen, yaitu OMOP concept ID dan pasangan vocabulary-code. Contoh untuk query Body weight adalah OMOP ID 3025315 dan kode `loinc:29463-7`. Kedua identifier tersebut menunjuk konsep yang sama dan tidak dihitung sebagai dua gold concept terpisah.

Gold set ini diturunkan dari `database_data` pada `data/input/mapping_templates.json`. Hasil yang dibandingkan terhadap gold set adalah kandidat Top-10 dalam `data/output/baseline/baseline-gemma3n-s`. Kandidat tersebut berasal dari dense retrieval SapBERT dan sparse retrieval BM42/FastEmbed pada collection `Qdrant_concept_mapping_1`, kemudian digabung secara interleaving dan dideduplikasi. Sebuah kandidat dinilai relevan apabila sekurang-kurangnya satu nilai pada `candidate.identifiers` sama dengan OMOP ID atau vocabulary-code pada `gold_ids`.

Secara ringkas, evaluasi melakukan pemeriksaan berikut untuk setiap kandidat:

```
relevan = candidate.identifiers gold_ids tidak kosong
```

Cara Menghitung Metrik

- **Accuracy@k** adalah proporsi query yang konsep benarnya ditemukan setidaknya sekali dalam Top-k. Nilai `Accuracy@1=0.60` berarti 6 dari 10 query mempunyai konsep benar pada rank pertama. `Accuracy@10=0.90` berarti konsep benar ditemukan dalam Top-10 untuk 9 dari 10 query.
- **Precision@k** adalah jumlah kandidat relevan dalam Top-k dibagi k. Karena benchmark ini hanya mempunyai satu konsep target per query, nilai precision secara alami mengecil ketika k bertambah. Oleh karena itu, `Accuracy@k` dan `Recall@k` lebih informatif untuk membaca keberhasilan candidate generation pada benchmark single-label ini.
- **Recall@k** adalah jumlah konsep gold yang ditemukan dalam Top-k dibagi jumlah konsep gold. Karena hanya ada satu konsep target per query, `Recall@k` pada eksperimen ini sama dengan `Accuracy@k`.
- **Reciprocal Rank (RR)** untuk satu query adalah $1/\text{rank}$ dari kandidat benar pertama. Query yang benar pada rank 1 mendapat RR 1, rank 2 mendapat 0.5, rank 5 mendapat 0.2, dan query tanpa kandidat benar mendapat 0. **MRR** adalah rata-rata RR seluruh query.
- **NDCG@k** memberi nilai lebih tinggi apabila kandidat benar berada lebih dekat ke rank pertama. Untuk satu konsep relevan, kontribusinya dihitung sebagai $1/\log_2(\text{rank}+1)$ dan dinormalisasi terhadap kondisi ideal saat konsep benar berada pada rank pertama.
- **Coverage** adalah jumlah query yang menghasilkan sekurang-kurangnya satu kandidat dibagi seluruh query.
- **Latency** diukur per query untuk dense Qdrant, sparse Qdrant, Athena, dan total retrieval. Laporan menyajikan mean, median (p50), dan persentil ke-95 (p95). Untuk hasil setelah

reranking, latency total juga mencakup pemanggilan Gemma.

Contoh: apabila konsep benar pertama muncul pada rank 3 dari lima kandidat, maka $\text{Accuracy@1}=0$, $\text{Accuracy@3}=1$, $\text{Precision@5}=1/5=0.2$, $\text{Recall@5}=1$, $\text{RR}=1/3$, dan $\text{NDCG@5}=1/\log_2(4)=0.5$ untuk query tersebut. Nilai akhir laporan adalah rata-rata hasil seluruh query.

Hasil Retrieval Hybrid

Retrieval hybrid Qdrant dense-sparse:

Metrik	Nilai
Coverage	1.00
Accuracy@1 / @3 / @5 / @10	0.60 / 0.60 / 0.70 / 0.90
Precision@1 / @3 / @5 / @10	0.60 / 0.20 / 0.14 / 0.09
Recall@1 / @3 / @5 / @10	0.60 / 0.60 / 0.70 / 0.90
MRR	0.64
NDCG@10	0.6965
Latency mean / p50 / p95	1488.18 / 1386.53 / 1959.57 ms

Coverage 1.00 dan $\text{query_failure_rate}=0.0$ menunjukkan seluruh query menghasilkan kandidat. Accuracy meningkat dari 0.60 pada Top-1 menjadi 0.90 pada Top-10, sehingga candidate generator mempunyai cakupan yang baik pada $k=10$, tetapi hanya enam query yang langsung benar pada posisi pertama. MRR 0.64 dan NDCG@10 0.6965 menunjukkan kandidat benar umumnya berada cukup tinggi, meskipun belum konsisten pada rank pertama.

Perbandingan Setelah Reranking Gemma

Setelah reranking Gemma:

Metrik	Nilai
Accuracy@1 / @3 / @5 / @10	0.40 / 0.70 / 0.80 / 0.90
MRR	0.5843
NDCG@10	0.6613
Reranking berhasil	10/10 query

Ground truth yang digunakan setelah reranking tetap sama; yang berubah hanya urutan kandidat. Gemma memperbaiki Accuracy@3 dari 0.60 menjadi 0.70 dan Accuracy@5 dari 0.70 menjadi 0.80, tetapi menurunkan Accuracy@1 dari 0.60 menjadi 0.40, MRR dari 0.64 menjadi 0.5843, serta NDCG@10 dari 0.6965 menjadi 0.6613. Artinya, Gemma berhasil membawa beberapa kandidat benar masuk ke Top-3 dan Top-5, tetapi juga memindahkan sejumlah kandidat yang semula benar pada rank pertama ke posisi yang lebih rendah. Karena itu reranking belum boleh diasumsikan selalu meningkatkan hasil dan memerlukan perbaikan prompt serta calibration study pada tahap berikutnya.

Keterbatasan Interpretasi

Gold set hanya terdiri atas 10 query dan diturunkan dari `mapping_templates.json`, yang juga berfungsi sebagai reservoir/training example baseline. Kondisi ini menimbulkan risiko data leakage

dan membuat sebagian query relatif mudah karena label query dekat atau identik dengan label konsep standar. Dataset ini juga belum merepresentasikan variasi istilah klinis Bahasa Indonesia, typo, singkatan ambigu, code-mixing, atau konteks klinis panjang, serta belum divalidasi secara independen oleh ahli terminologi klinis.

Dengan demikian, angka pada bagian ini harus diposisikan sebagai **internal smoke benchmark untuk reproducibility Tahap 0**, bukan sebagai estimasi performa final pipeline riset. Evaluasi ilmiah utama perlu menggunakan gold standard terpisah dari reservoir, mencakup variasi linguistik yang realistis, dan memperoleh validasi ahli.

Audit integrasi mengonfirmasi collection Qdrant `concept_mapping_1` tersedia dengan 3.695.703 point dan model Gemma dapat dipanggil melalui `TOGETHER_API_KEY`. Athena mengembalikan HTTP 403 untuk seluruh query. `partial_source_error_rate=1.0` mencatat gangguan tersebut, sedangkan `query_failure_rate=0.0` karena Qdrant tetap menghasilkan kandidat untuk semua query.

Status

Status: Tahap 0 selesai sebagai baseline reproducible dengan smoke audit, integration audit, dependency snapshot, output Top-k, manifest, full Gemma reranking, metrik, coverage, dan latency. Athena berstatus keterbatasan eksternal terdokumentasi; eksperimen memakai fallback Qdrant dense-sparse dan tidak menyamarkan sumber yang gagal sebagai passed.